UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 0. Real-bank slots: 11. Unofficial model answers:

   11/11.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (2).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Generation matching (Table 1):

1. High level language such as FORTRAN and COBOL were introduced and highly in demand → Second generation (2nd generation, c. 1959–1965). These languages appeared in the late 1950s and are characteristic of the transistor-based second generation.

2. Punched cards, paper tape and magnetic tape input and output devices were widely used → Second generation (2nd generation). These media are the classic I/O of second-generation machines (also used earlier, but the standard textbook answer for this characteristic is the second generation).

3. Hundred thousand of circuits integrated in a single chip where a concept of personal computing was being developed → Fourth generation (4th generation, VLSI/microprocessor era). Large-scale integration of circuits on a single chip and the rise of personal computers define the fourth generation.

4. Computers start to incorporate artificial intelligence capability to make decision based on prior knowledge → Fifth generation (5th generation). AI/knowledge-based decision making is the defining feature of fifth-generation computing.

5. Computers in this generation have a fast processing power with no requirement for air conditioning to cool the hardware → Fourth generation (4th generation). Microprocessor/VLSI machines gave high processing power in compact systems that no longer needed the special air-conditioned rooms of earlier valve-based generations.

Summary of answers: (1) 2nd, (2) 2nd, (3) 4th, (4) 5th, (5) 4th.

Unofficial marking guide:

- *1 mark: FORTRAN/COBOL → Second generation*
- *1 mark: Punched cards/paper tape/magnetic tape → Second generation*
- *1 mark: Hundred thousand circuits on one chip / personal computing → Fourth generation*
- *1 mark: Artificial intelligence / knowledge-based decisions → Fifth generation*
- *1 mark: Fast processing with no air conditioning → Fourth generation*

Model confidence: medium.

Model note: The table is given only as extracted text; each row is matched to the standard textbook generation characteristic. Row 2 (punched cards/paper tape/magnetic tape) is historically also used in first-generation machines, but the conventional CSC429 answer for this characteristic is the second generation; row 5 is likewise a standard fourth-generation descriptor. If the official scheme uses a different convention for these two rows, adjust accordingly.

QUESTION A2 (5 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (14).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Based on the Jacquard Loom excerpt, the Von Neumann input component is the punched cards (and the mechanism that reads them): the holes in each card encode the design row, so the card deck supplies the program/data to the loom. The output component is the woven textile (the patterned cloth produced by the loom). The loom itself corresponds to the processing unit, and the cards strung together in order correspond to stored instructions/data.

(b) Similarity: Both the mechanical calculating machine (e.g., Pascal's Pascaline or Leibniz's stepped reckoner) and the Atanasoff-Berry Computer (ABC) are special-purpose calculating devices designed to perform arithmetic automatically, and both use mechanical or electromechanical components to carry out computation. Dissimilarity: The mechanical calculating machine is purely mechanical and operates on decimal digits with no electronic memory or stored-program control, whereas the ABC is an electronic digital computer that uses vacuum tubes, binary arithmetic, and regenerative capacitor memory for intermediate storage; it also separates computing and memory units, unlike the purely mechanical calculator.

Unofficial marking guide:

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Correctly identifies the punched cards (or card reader) as the input component.
• 1 mark: Correctly identifies the woven textile/cloth (or the loom's output) as the output component.
• 1 mark: States one valid similarity between the mechanical calculating machine and the ABC (e.g., both are automatic calculating devices, both perform arithmetic).
• 2 marks: States one valid dissimilarity with explanation (e.g., mechanical/decimal vs electronic/binary, or absence vs presence of electronic memory/stored program). Award 1 mark for a bare dissimilarity and 2 marks for a clear, correct contrast.
Model confidence: medium.
Model note: The question refers to 'the mechanical calculating machine' without specifying which one; the answer assumes a generic early mechanical calculator such as Pascal's Pascaline. The Jacquard Loom excerpt does not explicitly name input/output components; the identification is inferred from the description of punched cards controlling the loom and the loom producing patterned textiles.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Source: CSC429 (18).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The bit string in Figure 1 is 40 bits long, so it forms 5 bytes. Grouping from the left in 8-bit units: 01010111 00111000 11110010 11001110 00011001. Converting each byte to hex: 01010111 = 57h, 00111000 = 38h, 11110010 = F2h, 11001110 = CEh, 00011001 = 19h. Hence the byte content is 57 38 F2 CE 19 (hex), i.e. the five bytes 0x57, 0x38, 0xF2, 0xCE, 0x19. (If interpreted as one 40-bit value it is 0x5738F2CE19.)

(b) Work in hex first, then convert to binary. 2D.5h = 0010 1101 . 0101 in binary (2 = 0010, D = 1101, .5 = 0101). ED.2h = 1110 1101 . 0010 in binary (E = 1110, D = 1101, .2 = 0010). Adding the fractional parts: 0101 + 0010 = 0111 (no carry). Adding the integer parts: 0010 1101 + 1110 1101. Low nibble: D + D = 1101 + 1101 = 1 1010 = 26 decimal, i.e. A with carry 1. High nibble: 2 + E + carry 1 = 0010 + 1110 + 0001 = 1 0001 = 17 decimal, i.e. 1 with carry 1 out. So the integer sum is 1 0001 1010 = 11Ah. Therefore 2D.5h + ED.2h = 11A.7h = 1 0001 1010 . 0111 in binary, i.e. 100011010.0111_2. Check in decimal: 2D.5h = 45.3125, ED.2h = 237.125, sum = 282.4375 = 11A.7h, confirming the result.

Unofficial marking guide:

• 1 mark: correctly groups the 40-bit string into five 8-bit bytes (01010111 00111000 11110010 11001110 00011001)
• 1 mark: correctly converts all five bytes to hex giving 57 38 F2 CE 19
• 1 mark: correctly converts 2D.5h and ED.2h to binary (00101101.0101 and 11101101.0010)
• 1 mark: correct binary/hex addition with carries shown, giving 11A.7h
• 1 mark: correct final binary answer 100011010.0111_2 (with working/check shown)
Model confidence: high.
Model note: Figure 1 is assumed to be the 40-bit string given in the question text; if the actual figure differs, the byte grouping must be read from it. The hex suffix 'h' is treated as standard hexadecimal notation.

QUESTION A4 (5 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (10).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A cache is a small, fast, high-cost memory that sits between the CPU and main memory (or between two levels of the memory hierarchy) and holds copies of the most recently or most frequently used blocks of data and instructions. Because it is faster than main memory, the CPU can satisfy most accesses from the cache, reducing the average memory access time and hiding the speed gap between the fast processor and slower main memory. It exploits the principles of locality of reference (temporal and spatial locality).

(b) Two ways to store (map) data in cache:

1. Direct mapping: each main-memory block is mapped to exactly one fixed cache line, computed as cache line = (block address) mod (number of cache lines). The cache line stores the data block plus a tag (the remaining high-order address bits) and a valid bit. It is simple and cheap to search (only one line is checked), but it suffers from conflict misses when two frequently used blocks map to the same line.

2. Set-associative mapping: the cache is divided into sets, each containing N lines (N-way set associative). A main-memory block maps to a fixed set, set = (block address) mod (number of sets), but can be placed in any one of the N lines within that set. The tag is compared against all N lines in the set

© Hak Cipta Universiti Teknologi MARA

in parallel. This reduces conflict misses compared with direct mapping while keeping the search cheaper than fully associative mapping. (Fully associative mapping, where a block may go into any cache line and all tags are compared in parallel, is an acceptable alternative second way.)

Unofficial marking guide:

• 1 mark: (a) correct definition of cache as a small, fast memory holding recently/frequently used data between CPU and main memory to reduce average access time (locality of reference).

• 2 marks: (b) first method correctly named (e.g. direct mapping) with a correct description of how a block is placed (mapping formula/one fixed line) and the role of the tag.

• 2 marks: (b) second method correctly named (e.g. set-associative or fully associative mapping) with a correct description of how a block is placed (fixed set, any line in set; or any line at all) and the tag comparison.

Model confidence: high.

Model note: Question is generic; no figure or table required. 'Ways to store data in cache' is interpreted as cache mapping/organisation schemes (direct, set-associative, fully associative), which is the standard CSC429 interpretation.

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (1).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) X is the system bus (also called the computer bus / interconnection network). In Figure 1 it is the set of parallel lines linking the main memory, the CPU (the 'Computer Systems' block) and the peripherals (input, output and communication lines).

(b) The function of X is to provide a shared communication pathway that carries data, addresses and control signals between the processor, main memory and I/O devices. It allows the CPU to fetch instructions and operands from memory, write results back, and transfer data to and from peripherals, so that the otherwise separate components can cooperate as one working computer system.

(c) The sub-structural components of the system bus are: (i) the data bus — carries the actual data/instructions between components, its width determining the number of bits transferred at once; (ii) the address bus — carries the address of the memory location or I/O port being accessed, its width determining the addressable memory space; and (iii) the control bus — carries control and timing signals such as read/write, memory/IO select, clock and interrupt/acknowledge lines. (Some texts also list power/ground and arbitration lines.)

In my opinion X is very important because it is the single shared medium through which all components communicate; without it the CPU, memory and peripherals could not exchange data, so the system could not function. It also determines key performance limits: the bus width and clock speed set how much data can be moved per unit time, and because it is shared, bus contention and arbitration directly affect overall system throughput and speed.

(d) No separate question text is given for part (d); it is treated as the concluding 'why X is important' opinion asked in part (c). If the paper intends a distinct 2-mark point, the expected answer is that X is the communication backbone / bottleneck of the system: it interconnects all units and its bandwidth and speed govern overall system performance.

Unofficial marking guide:

• 2 marks: correctly identifies X as the system bus / interconnection bus (1 mark for 'bus', 1 mark for 'system/interconnection' qualifier).

• 2 marks: states it is a shared communication pathway (1 mark) and that it carries data/address/control between CPU, memory and I/O (1 mark).

• 4 marks: lists data bus (1), address bus (1), control bus (1); 1 mark for a reasoned statement of importance (shared medium / performance bottleneck).

• 2 marks: explains importance as the communication backbone whose width/speed limits overall system performance (1 mark for backbone role, 1 mark for performance/bandwidth point).

Model confidence: medium.

Model note: Figure 1 is only available as extracted text; X is inferred to be the system bus from the layout (Main Memory, Computer Systems/CPU, Peripherals joined by interconnection lines). Part (d) has no question text in the paper extract, so it is answered as the concluding importance/opinion point; if the original part (d) asked something else, the last 2 marks should be reallocated.

QUESTION A6 (10 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (8).pdf, page(s) 5

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The figure shows an instruction with an Opcode field, an Address A field, a Register field and an Operand field, where the operand is obtained from a register. This is register indirect addressing (more generally, a register-based indirect mode): the instruction specifies a register, and the contents of that register give the effective address of the operand. (If the figure is read as the register itself holding the operand, it would be register addressing; the presence of the separate 'Address A'/'Register' fields and the operand arrow indicates the register supplies the address, i.e. register indirect.)

(b) The register involved is a general-purpose CPU register (e.g. an index/base register such as R1, or the accumulator depending on the architecture). The register field in the instruction selects which CPU register holds the effective address of the operand; no memory access is needed to fetch the address itself.

(c) If the register is replaced by a memory location, the instruction's address field points to a memory word that contains the effective address of the operand. This is memory indirect addressing (indirect addressing via memory).

(d) The register indirect mode in (a) executes faster. Justification: in register indirect addressing the effective address is already held in a CPU register, so the operand fetch requires only one memory access (to read the operand). In memory indirect addressing the processor must first read the memory word containing the effective address (one memory access) and then read the operand (a second memory access), i.e. two memory accesses. Since memory access is much slower than register access (registers are inside the CPU and accessed in a fraction of a cycle), the extra memory reference makes memory indirect addressing slower. Hence register indirect addressing is faster.

Unofficial marking guide:

• 2 marks: (a) correctly identifies register indirect addressing (1 mark for 'indirect', 1 mark for 'via register').

• 2 marks: (b) states a general-purpose/CPU register (e.g. index or base register) holds the effective address (2 marks; 1 mark if only 'a register' without explanation).

• 2 marks: (c) correctly identifies memory indirect addressing (2 marks; 1 mark for partial/indirect only).

• 4 marks: (d) states register indirect is faster (1 mark); explains register access is faster than memory access (1 mark); explains register indirect needs one memory access vs two for memory indirect (1 mark); clear coherent justification/conclusion (1 mark).

Model confidence: medium.

Model note: Figure 2 is only available as extracted text (Opcode, Address A, Register, Operand). The answer assumes the standard interpretation that the register supplies the effective address (register indirect). If the figure actually shows the register holding the operand directly, (a) would be register addressing and (c) direct addressing; the speed comparison conclusion (register faster than memory) still holds. Marker should confirm the arrow/field layout from the original figure.

QUESTION A7 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (15).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) An interrupt is a signal (either generated by hardware — e.g. an I/O device completing a transfer or an error condition — or by software, such as a trap or system call) that causes the CPU to suspend the program it is currently executing, save the current context (program counter and register state), and transfer control to a special routine called an interrupt handler (interrupt service routine). When the handler finishes, the saved context is restored and the interrupted program resumes. Interrupts allow the CPU to respond promptly to asynchronous events instead of polling.

(b) Based on Figure 1, the interrupts are ranked by priority from highest to lowest as: Interrupt Handler W → Interrupt Handler X → Interrupt Handler Z → Interrupt Handler Y. This ordering follows the nesting shown in the figure: the highest-priority interrupt (W) is serviced first and may pre-empt the others; while W is being handled, a lower-priority interrupt (X) is held pending; Z is serviced next, and Y, the lowest-priority interrupt, is serviced last (after the higher-priority handlers complete).

(c) If interrupts are disabled (interrupt disable applied), the CPU masks further interrupts while it is servicing the current one, so the handlers are executed strictly in the order in which the interrupts arrive — no pre-emption/nesting occurs. Reading the arrival sequence from Figure 1, the order of service (most important to least important in terms of being handled first) is: Interrupt Handler W → Interrupt Handler X → Interrupt Handler Z → Interrupt Handler Y, i.e. the same sequence, but now each handler runs to completion before the next is started because interrupts are disabled.

Unofficial marking guide:

© Hak Cipta Universiti Teknologi MARA

• 1 mark: states that an interrupt is a signal/event (hardware or software) that diverts the CPU from its current program.
• 1 mark: states that the CPU saves its context and transfers control to an interrupt handler/service routine, then resumes the interrupted program afterwards.
• 2 marks: correctly identifies the priority order W > X > Z > Y (highest to lowest).
• 2 marks: justifies the order using the nesting/pre-emption shown in Figure 1 (higher-priority handler pre-empts lower-priority ones).
• 2 marks: states that with interrupts disabled no pre-emption/nesting occurs and handlers run to completion in arrival order.
• 2 marks: gives the resulting service order W → X → Z → Y consistent with the figure's arrival sequence.
Model confidence: medium.
Model note: Figure 1 is only available as extracted text; the exact nesting/arrival sequence of the handlers must be read from the figure. The answer assumes the vertical/nesting layout in Figure 1 places W highest and Y lowest, and that the arrival order is W, X, Z, Y. If the figure shows a different arrival order, part (c)'s sequence should follow that order.

QUESTION A8 (10 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (17).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The four classifications of parallel processors (Flynn's taxonomy) are:

1. SISD (Single Instruction, Single Data): one instruction stream operates on one data stream per clock cycle. Example: a conventional single-core von Neumann processor, such as an early Intel 8086 or a basic scalar CPU.

2. SIMD (Single Instruction, Multiple Data): one instruction operates on multiple data elements simultaneously using multiple processing elements. Example: vector processors (e.g., Cray-1) and modern GPU/SIMD extensions such as Intel SSE/AVX or ARM NEON.

3. MISD (Multiple Instruction, Single Data): multiple instruction streams operate on the same data stream. This class is rarely used in practice; a theoretical example is a systolic array or a fault-tolerant system with replicated processors processing the same input.

4. MIMD (Multiple Instruction, Multiple Data): multiple processors execute different instruction streams on different data streams. Examples: multicore CPUs (e.g., Intel Core i7), multiprocessor servers, and clusters of workstations.

(b) One advantage of cluster computer architecture is high availability / fault tolerance through redundancy: if one node in the cluster fails, the remaining nodes can continue providing service (failover), so the system as a whole is more reliable than a single computer. Another valid advantage is cost-effective scalability: commodity nodes can be added to increase performance at lower cost than a large symmetric multiprocessor. (Any one advantage discussed earns the marks.)

Unofficial marking guide:

• 1 mark: correctly naming SISD and giving a valid example
• 1 mark: correctly naming SIMD and giving a valid example
• 1 mark: correctly naming MISD and giving a valid example
• 1 mark: correctly naming MIMD and giving a valid example
• 4 marks: accurate description/definition of each of the four classifications (1 mark each)
• 2 marks: one valid advantage of cluster computing clearly discussed (e.g., high availability/fault tolerance or cost-effective scalability)
Model confidence: high.
Model note: The question asks for 'FOUR classifications of parallel processors'; Flynn's taxonomy is assumed as the standard classification. MISD is included as one of the four, although it is rarely implemented. For part (b), any one reasonable advantage is acceptable; the model answer gives two common ones but only one is required.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (13).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two different types of bus: (1) The data bus, which carries the actual data/operands between the CPU, memory and I/O; its width determines how many bits can be transferred at once. (2) The address bus, which carries the memory or I/O address selected by the CPU; its width determines the size of the

© Hak Cipta Universiti Teknologi MARA

addressable memory space. (Other acceptable answers: control bus, system bus, internal/CPU bus, expansion bus such as PCI.)

(b) Given: word = 32 bits, block = 256 bits.

(i) Bytes per word: 1 byte = 8 bits, so bytes per word = 32 ÷ 8 = 4 bytes per word.

(ii) Bytes per block: bytes per block = 256 ÷ 8 = 32 bytes per block. (Equivalently, the block holds 256 ÷ 32 = 8 words.)

(iii) Size of the register: the register is word-sized, so the register size = word size = 32 bits = 4 bytes (32 bits).

(iv) Size of the data bus: the data bus must transfer one word per access, so the data bus width = word size = 32 bits (4 bytes). (If the design transferred a whole block per access the data bus would be 256 bits, but the standard word-organised assumption gives 32 bits.)

Unofficial marking guide:

- *1 mark: correctly naming the data bus (with a brief correct description)*
- *1 mark: correctly naming a second distinct bus, e.g. address bus or control bus*
- *2 marks: bytes per word = 32/8 = 4 bytes (1 mark for the division by 8, 1 mark for the correct answer)*
- *2 marks: bytes per block = 256/8 = 32 bytes (1 mark for method, 1 mark for correct answer)*
- *2 marks: register size = 32 bits = 4 bytes, justified as equal to the word size*
- *2 marks: data bus size = 32 bits = 4 bytes, justified as equal to the word size*

Model confidence: high.

Model note: Assumes the standard convention that the register and data bus are word-sized (one word transferred per memory access). If the lecturer intends the data bus to match the block size, part (iv) would be 256 bits; the word-based answer is given as the primary answer. No figure is required.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

Source: CSC429 (16).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Convert -156.15625_{10} to IEEE 754 single precision (hex).

Step 1 — Sign: the number is negative, so S = 1.

Step 2 — Convert magnitude to binary. 156 = 128 + 16 + 8 + 4 = 10011100_2. For the fraction: 0.15625 × 2 = 0.3125 (0); 0.3125 × 2 = 0.625 (0); 0.625 × 2 = 1.25 (1); 0.25 × 2 = 0.5 (0); 0.5 × 2 = 1.0 (1). So $0.15625_{10} = 0.00101_2$. Hence $156.15625_{10} = 10011100.00101_2$.

Step 3 — Normalise: $10011100.00101_2 = 1.001110000101_2 \times 2^7$. So the exponent field value is 7.

Step 4 — Biased exponent: E = 7 + 127 = 134 = 10000110_2.

Step 5 — Mantissa (fraction after the leading 1): 001110000101, padded to 23 bits: 0011 1000 0101 0000 0000 000.

Step 6 — Assemble: S | E | M = 1 10000110 00111000010100000000000.
Group into nibbles: 1100 0011 0001 1100 0010 1000 0000 0000 = C31C2800H.

Answer: -156.15625_{10} = C31C2800H (IEEE 754 single precision).

(b) Convert BD40 0000H to decimal.

Step 1 — Expand hex to binary: B D 4 0 0 0 0 0 = 1011 1101 0100 0000 0000 0000 0000 0000.

Step 2 — Split fields: S = 1 (negative). Exponent = 0111 $1010_2 = 122_{10}$. Mantissa = 100 0000 0000 0000 0000 0000_2.

Step 3 — Unbias exponent: e = 122 − 127 = −5.

Step 4 — Reconstruct significand: 1.M = $1.1000\ldots0_2 = 1.5_{10}$ (since the mantissa is 100…0 = 0.5).

Step 5 — Apply exponent: value = $-1.5 \times 2^{-5} = -1.5 \times 0.03125 = -0.046875$.

Answer: BD40 0000H = -0.046875_{10}.

Unofficial marking guide:

- *1 mark: correct sign bit S = 1 for the negative number in (a)*
- *1 mark: correct binary conversion of 156.15625 to 10011100.00101_2*
- *1 mark: correct normalisation $1.001110000101 \times 2^7$ and biased exponent 134 = 10000110_2*
- *1 mark: correct 23-bit mantissa field 00111000010100000000000*

© Hak Cipta Universiti Teknologi MARA

- *1 mark: correct final hex C31C2800H in (a)*
- *1 mark: correct binary expansion of BD40 0000H and field split (S=1, E=01111010, M=100…0)*
- *1 mark: correct unbiased exponent e = 122 − 127 = −5*
- *1 mark: correct significand 1.5 and computation -1.5×2^{-5}*
- *1 mark: correct final decimal −0.046875 in (b)*
- *1 mark: clear workflow shown for both parts (working steps visible)*

Model confidence: high.

Model note: OCR rendered the subscript 10 as '10' in '-156.1562510' and 'BD40 0000H'; interpreted as decimal -156.15625 and hex BD40 0000H respectively. The value 0.15625 is exactly representable, so no rounding is needed. Final hex C31C2800H verified by re-expansion.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (21).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 5 reads z-A+c2/B*D-3A. The extracted text is garbled; the standard form of this classic question is taken to be z = A + (C × 2) / (B × D) − 3A, i.e. z = A + C*2/B*D - 3A. If the figure actually shows a different arrangement (e.g. C^2 or C*2), the same instruction sequences apply with the operands reordered; the structure of the answer is unchanged. Registers A, B, C, D are fixed; x, y, z are scratch; result must end in z.

Zero-address (stack) program. All operands are pushed onto the stack; operations pop their operands and push the result. Evaluate left-to-right with the usual precedence (× and / before + and −):

```
PUSH A        ; stack: A
PUSH C
PUSH 2
MUL           ; C*2
PUSH B
PUSH D
MUL           ; B*D
DIV           ; (C*2)/(B*D)
ADD           ; A + (C*2)/(B*D)
PUSH 3
PUSH A
MUL           ; 3A
SUB           ; A + (C*2)/(B*D) - 3A
POP z         ; store final answer in z
```

One-address program. One operand is implicit in the accumulator; the named operand is the memory/register operand. Using x and y as temporaries:

```
LOAD C
MUL 2         ; ACC = C*2
STOR x        ; x = C*2
LOAD B
MUL D         ; ACC = B*D
STOR y        ; y = B*D
LOAD x
DIV y         ; ACC = (C*2)/(B*D)
ADD A         ; ACC = A + (C*2)/(B*D)
STOR x        ; save partial result in x
LOAD 3
MUL A         ; ACC = 3A
STOR y        ; y = 3A
LOAD x
SUB y         ; ACC = A + (C*2)/(B*D) - 3A
STOR z        ; final answer in z
```

Three-address program. Each instruction is OP (dest, src1, src2); temporaries x and y hold intermediate results:

```
MUL (x, C, 2)      ; x = C*2
MUL (y, B, D)      ; y = B*D
DIV (x, x, y)      ; x = (C*2)/(B*D)
ADD (x, A, x)      ; x = A + (C*2)/(B*D)
MUL (y, 3, A)      ; y = 3A
SUB (z, x, y)      ; z = A + (C*2)/(B*D) - 3A
```

In all three versions the final value is placed in register z, and the fixed registers A, B, C, D are never overwritten (only x and y are used as scratch).

© Hak Cipta Universiti Teknologi MARA

Unofficial marking guide:

• 4 marks: correct zero-address program — correct PUSH/POP ordering, correct use of MUL/DIV/ADD/SUB, result POPped into z

• 4 marks: correct one-address program — correct LOAD/STOR/MUL/DIV/ADD/SUB with accumulator, temporaries x and y used correctly, result STORed to z

• 4 marks: correct three-address program — correct three-operand MUL/DIV/ADD/SUB form, temporaries used, result written to z

• 4 marks: correct evaluation order/precedence (× and ÷ before + and −), correct handling of the 3A term, and no corruption of fixed registers A, B, C, D

• 4 marks: overall correctness and consistency across the three programs (same computed expression, final answer in z in every case)

Model confidence: medium.

*Model note: The equation in Figure 5 is OCR-garbled ('z-A+c2/B*D-3A'); I conservatively interpreted it as z = A + C*2/(B*D) − 3A, the standard form of this question. If the figure shows C² or a different grouping, the operand order changes but the instruction structure and marking allocation remain valid. The exact register/operand syntax of the three-address form is taken from Table 1 as OP(p,q,r).*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA